

lecture_26

October 2, 2026

1 Lecture 26: Simulated Annealing - Benchmarking

{note} Lecture 25 ran Simulated Annealing on the Ackley function - the simplest possible setting, chosen so the lecture could focus entirely on turning pseudocode into code. This lecture moves to what this module actually cares about: real routing benchmarks from **TSPLIB**, the standard instance library the metaheuristics literature itself uses. It calibrates SA's cooling schedule on a small TSPLIB instance, then asks the question that actually matters in practice - does that calibration still work once you deploy on a substantially larger instance?

Learning Objectives

By the end of this notebook, you will be able to: 1. Represent a TSPLIB instance and implement a 2-opt neighbourhood for it. 2. Calibrate Simulated Annealing's cooling schedule on a small instance under a limited budget. 3. Evaluate whether a calibration found on a small instance still performs well on a substantially larger one, and explain why a cooling *rate* does not automatically rescale with problem size.

Prerequisites: Simulated Annealing - Algorithm (Lecture 25).

Estimated time: 50 minutes

1.1 Why TSPLIB?

TSPLIB (Reinelt, 1991) is a public library of Traveling Salesman Problem instances — many drawn from real geographic data — used throughout the metaheuristics literature as a common, credible benchmark. Unlike the Chennai Metro Feeder Route (this module's own small illustrative instance, reserved for Lecture 33's capacitated capstone), TSPLIB instances come with **published optimal tour lengths**, so a result can be reported as a precise percentage gap from the true optimum rather than “seems reasonable.”

This lecture uses two instances from the same family (Christofides & Eilon): - **eil51** — 51 stops, optimal tour length **426** — small enough that a 10-seed calibration sweep stays fast. - **eil101** — 101 stops, optimal tour length **629** — roughly twice the combinatorial scale, used only to test whether a calibration found on **eil51** still holds.

Since a solution is now a **tour** rather than a point in $\mathbb{R}^2$, this lecture also introduces the neighbourhood Lecture 25 never needed: **2-opt**, which reverses a randomly chosen segment of the tour — the same move this lecture’s pre-restructuring predecessor used for the Chennai TSP, now meeting Simulated Annealing for the first time in this arc.

```
[86]: import random
import numpy as np
import matplotlib.pyplot as plt

def sa(s_o, f, N, T_o, r, n=2000, t=1e-6):
    """Simulated Annealing.

    Parameters
    -----

    s_o : initial solution
    f   : objective function, f(s) -> float (minimized)
    N   : neighbourhood function, N(s) -> a random neighbour of s
    T_o : initial temperature
    r   : cooling rate (T_k = T_o * r**k)
    n   : maximum number of iterations
    t   : convergence threshold on improvement in the best solution

    Returns
    -----

    s_b : best solution found
    F_c : current objective value, after each iteration (for plotting)
    F_b : best objective value found, after each iteration (for plotting)
    """

    s_c = s_o                # current solution
    s_b = s_c                # best solution found so far
    F_c = [f(s_c)]
    F_b = [f(s_b)]
    T   = T_o                # temperature
    k, e, converged = 0, float("inf"), False
    while not converged:
        s_n = N(s_c)          # random neighbour
        d = f(s_n) - f(s_c)    # change in objective
        if d < 0:              # always accept an improving move
            s_c = s_n
        elif random.uniform(0, 1) < np.exp(-d / T): # Boltzmann acceptance
            s_c = s_n
        if f(s_c) < f(s_b):
            e = f(s_b) - f(s_c)
            s_b = s_c
        F_c.append(f(s_c))
        F_b.append(f(s_b))
```

```

    k += 1
    T = T_o * r ** k
    if k >= n or e <= t:
        converged = True
    return s_b, F_c, F_b

```

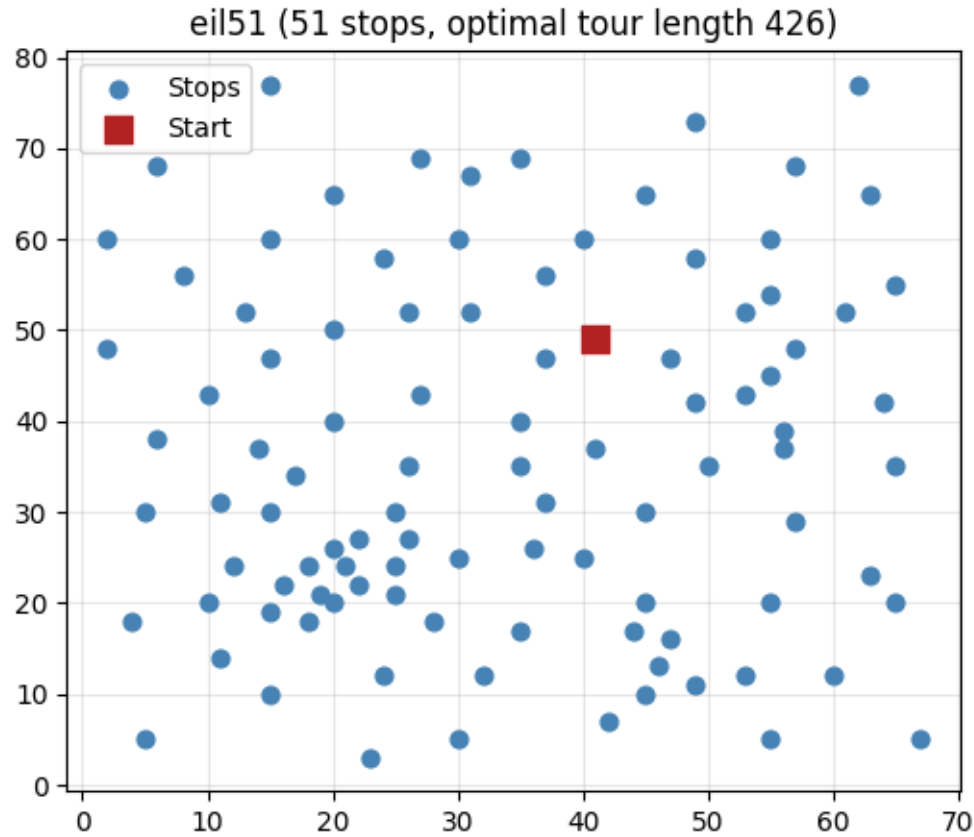
```

[87]: # eil51: 51-city problem
coords = [
    (37, 52), (49, 49), (52, 64), (20, 26), (40, 30), (21, 47), (17, 63), (31, ↵
    ↵62), (52, 33), (51, 21),
    (42, 41), (31, 32), (5, 25), (12, 42), (36, 16), (52, 41), (27, 23), (17, ↵
    ↵33), (13, 13), (57, 58),
    (62, 42), (42, 57), (16, 57), (8, 52), (7, 38), (27, 68), (30, 48), (43, ↵
    ↵67), (58, 48), (58, 27),
    (37, 69), (38, 46), (46, 10), (61, 33), (62, 63), (63, 69), (32, 22), (45, ↵
    ↵35), (59, 15), (5, 6),
    (10, 17), (21, 10), (5, 64), (30, 15), (39, 10), (32, 39), (25, 32), (25, ↵
    ↵55), (48, 28), (56, 37),
    (30, 40),
]
depot = 0
customers = list(range(1, len(coords)))
optimal = 426

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(X[1:], Y[1:], c="steelblue", s=40, zorder=2, label="Stops")
ax.scatter(X[0], Y[0], c="firebrick", marker="s", s=100, zorder=3, ↵
    ↵label="Start")
ax.set_title("eil51 (51 stops, optimal tour length 426)")
ax.legend()
ax.grid(alpha=0.3)
plt.show()

n = len(coords)
X = np.array([c[0] for c in coords], dtype=float)
Y = np.array([c[1] for c in coords], dtype=float)
D = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        D[i, j] = np.hypot(X[i] - X[j], Y[i] - Y[j])

```



```
[88]: def f(s):
    c = D[0, s[0]]
    for i in range(len(s) - 1):
        c += D[s[i], s[i + 1]]
    c += D[s[-1], 0]
    return c

def N(s):
    """2-opt neighbourhood: reverse a random segment of the tour."""
    s = list(s)
    i, j = sorted(random.sample(range(len(s)), 2))
    return tuple(s[:i] + s[i:j + 1][::-1] + s[j + 1:])
```

1.2 Calibrating the Cooling Rate

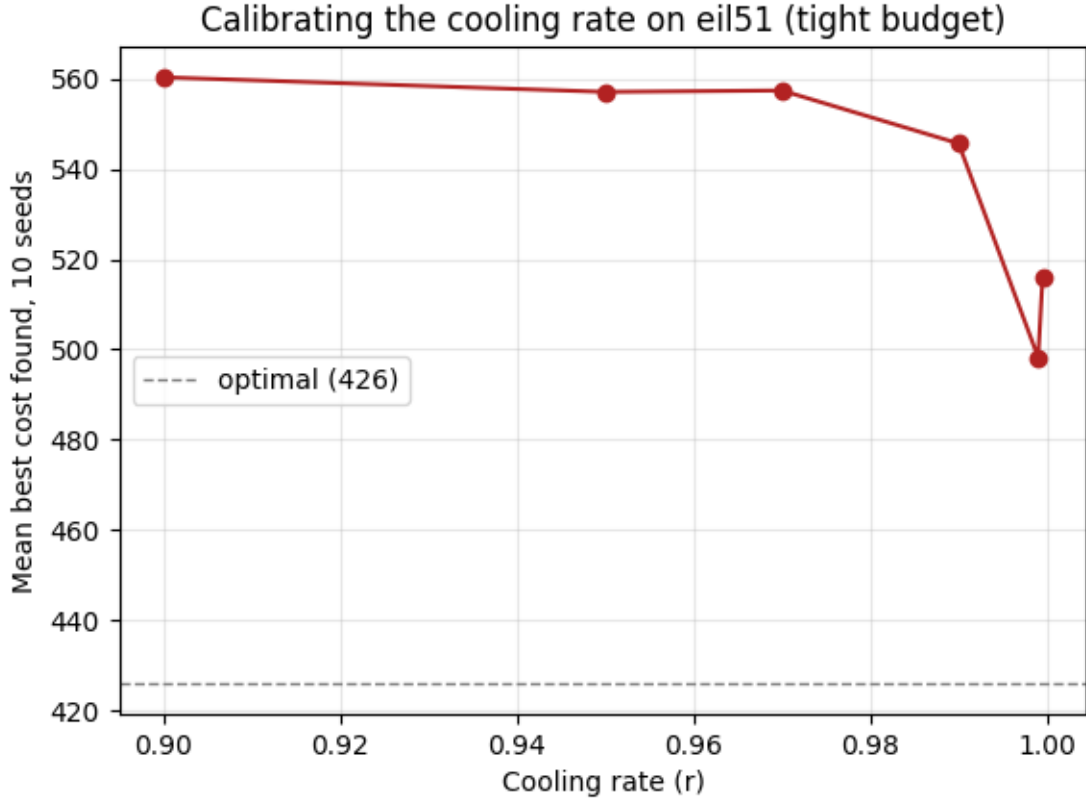
Holding $T_o = 20$ fixed and a tight budget of 1,000 iterations, only the cooling rate r changes — averaged over 30 seeds per configuration, the same practice this module has used throughout.

```
[91]: R = {}
for r in [0.9, 0.95, 0.97, 0.99, 0.999, 0.9995]:
    Z = []
    for seed in range(1, 31):
        random.seed(seed)
        s_o = tuple(random.sample(customers, len(customers)))
        s_b, F_c, F_b = sa(s_o, f, N, 20, r, 10000, 1e-12)
        z = f(s_b)
        Z.append(z)
    R[r] = Z
    gap = (np.mean(Z) - optimal) / optimal * 100
    print(f"r={r:<6} -> mean cost = {np.mean(Z):.2f}    gap from optimal = {gap:.1f}%")

fig, ax = plt.subplots(figsize=(6.5, 4.5))
ax.plot(list(R.keys()), [np.mean(v) for v in R.values()], "o-",
        color="firebrick")
ax.axhline(optimal, color="gray", linestyle="--", linewidth=1, label="optimal",
        color="red")
ax.set_xlabel("Cooling rate (r)")
ax.set_ylabel("Mean best cost found, 10 seeds")
ax.set_title("Calibrating the cooling rate on eil51 (tight budget)")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```

```
/var/folders/_0/vp0g6w3s2jg_2m0m2ljwgy40000gn/T/ipykernel_85513/902333111.py:35
: RuntimeWarning: overflow encountered in scalar divide
    elif random.uniform(0, 1) < np.exp(-d / T):    # Boltzmann acceptance
/var/folders/_0/vp0g6w3s2jg_2m0m2ljwgy40000gn/T/ipykernel_85513/902333111.py:35
: RuntimeWarning: divide by zero encountered in scalar divide
    elif random.uniform(0, 1) < np.exp(-d / T):    # Boltzmann acceptance
/var/folders/_0/vp0g6w3s2jg_2m0m2ljwgy40000gn/T/ipykernel_85513/902333111.py:35
: RuntimeWarning: invalid value encountered in scalar divide
    elif random.uniform(0, 1) < np.exp(-d / T):    # Boltzmann acceptance

r=0.9      -> mean cost = 560.36    gap from optimal = 31.5%
r=0.95     -> mean cost = 557.12    gap from optimal = 30.8%
r=0.97     -> mean cost = 557.43    gap from optimal = 30.9%
r=0.99     -> mean cost = 545.68    gap from optimal = 28.1%
r=0.999    -> mean cost = 498.25    gap from optimal = 17.0%
r=0.9995   -> mean cost = 515.94    gap from optimal = 21.1%
```



1.3 Benchmarking at Scale

eil101 has roughly twice as many stops as eil51. Apply the calibrated schedule ($T_o = 20$, $r = 0.97$) **exactly as found**, at the same 1,000-iteration budget, and see what happens.

```
[92]: # eil101: 101-city problem
coords = [
    (41, 49), (35, 17), (55, 45), (55, 20), (15, 30), (25, 30), (20, 50), (10, 43),
    (55, 60), (30, 60),
    (20, 65), (50, 35), (30, 25), (15, 10), (30, 5), (10, 20), (5, 30), (20, 40),
    (15, 60), (45, 65),
    (45, 20), (45, 10), (55, 5), (65, 35), (65, 20), (45, 30), (35, 40), (41, 37),
    (64, 42), (40, 60),
    (31, 52), (35, 69), (53, 52), (65, 55), (63, 65), (2, 60), (20, 20), (5, 5),
    (60, 12), (40, 25),
    (42, 7), (24, 12), (23, 3), (11, 14), (6, 38), (2, 48), (8, 56), (13, 52),
    (6, 68), (47, 47),
    (49, 58), (27, 43), (37, 31), (57, 29), (63, 23), (53, 12), (32, 12), (36, 26),
    (21, 24), (17, 34),
```

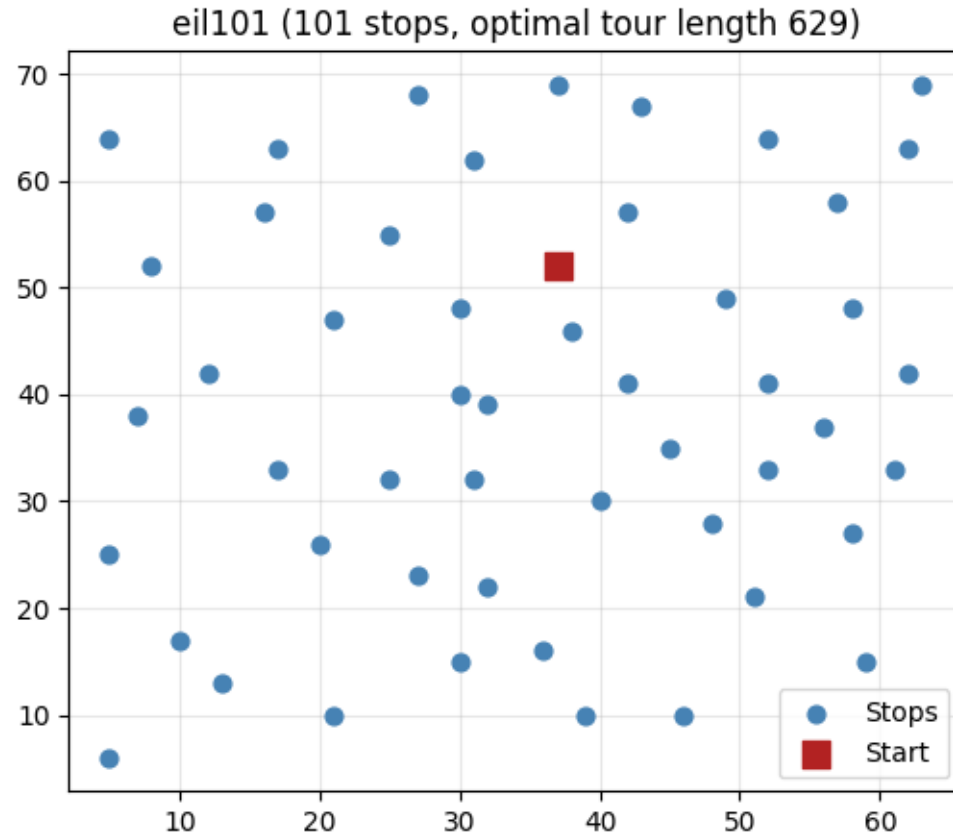
```

    (12, 24), (24, 58), (27, 69), (15, 77), (62, 77), (49, 73), (67, 5), (56,
↪39), (37, 47), (37, 56),
    (57, 68), (47, 16), (44, 17), (46, 13), (49, 11), (49, 42), (53, 43), (61,
↪52), (57, 48), (56, 37),
    (55, 54), (15, 47), (14, 37), (11, 31), (16, 22), (4, 18), (28, 18), (26,
↪52), (26, 35), (31, 67),
    (15, 19), (22, 22), (18, 24), (26, 27), (25, 24), (22, 27), (25, 21), (19,
↪21), (20, 26), (18, 18),
    (35, 35),
]
depot = 0
customers = list(range(1, len(coords)))
optimal = 629

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(X[1:], Y[1:], c="steelblue", s=40, zorder=2, label="Stops")
ax.scatter(X[0], Y[0], c="firebrick", marker="s", s=100, zorder=3,
↪label="Start")
ax.set_title("eil101 (101 stops, optimal tour length 629)")
ax.legend()
ax.grid(alpha=0.3)
plt.show()

n = len(coords)
X = np.array([c[0] for c in coords], dtype=float)
Y = np.array([c[1] for c in coords], dtype=float)
D = np.zeros((n, n))
for i in range(n):
    for j in range(n):
        D[i, j] = np.hypot(X[i] - X[j], Y[i] - Y[j])

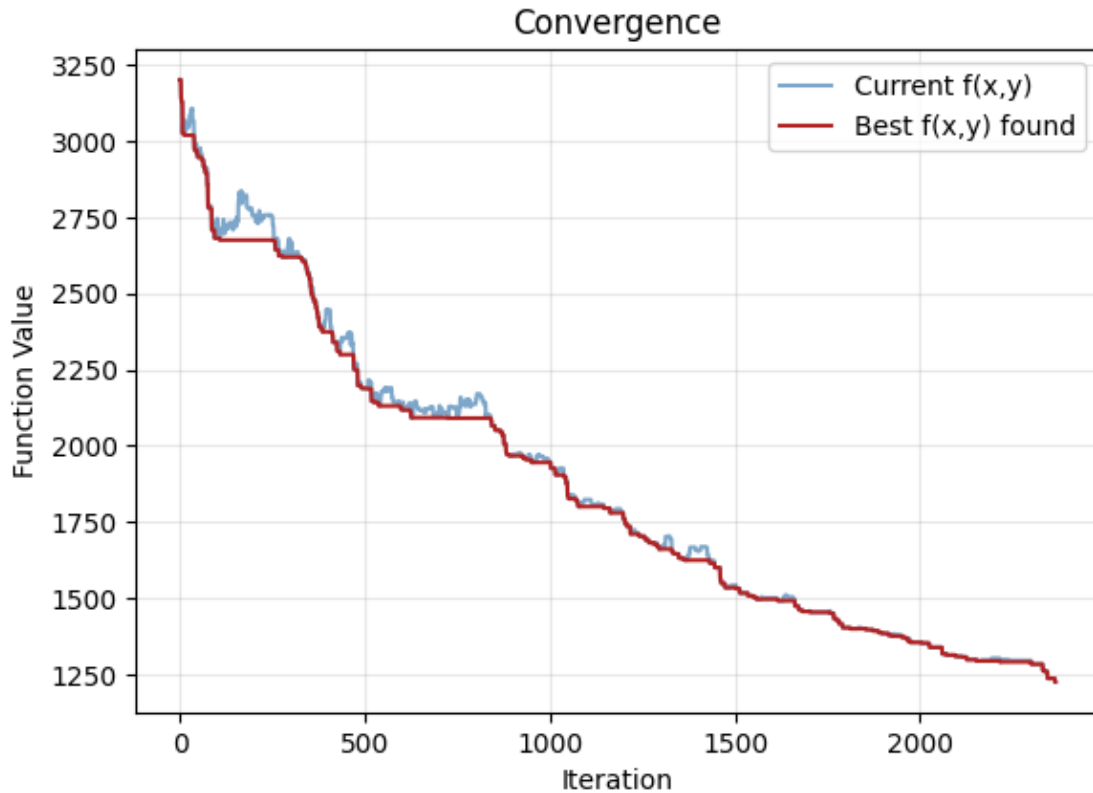
```



```
[117]: random.seed(5972)
s_o = tuple(random.sample(customers, len(customers)))
s_b, F_c, F_b = sa(s_o, f, N, T_o=20, r=0.999, n=10000, t=1e-9)
z = f(s_b)
print(f"Naive transfer (T_o=20, r=0.999, n=10000): cost = {z:.2f}    "
      f"gap = {(z - optimal) / optimal * 100:.1f}%")

fig, ax = plt.subplots(figsize=(6.5, 4.5))
ax.plot(F_c, color="steelblue", linewidth=1.5, alpha=0.7, label="Current_
↳f(x,y)")
ax.plot(F_b, color="firebrick", linewidth=1.5, label="Best f(x,y) found")
ax.set_xlabel("Iteration")
ax.set_ylabel("Function Value")
ax.set_title("Convergence")
ax.grid(alpha=0.3)
ax.legend()
plt.show()
```

Naive transfer (T_o=20, r=0.999, n=10000): cost = 1224.70 gap = 94.7%



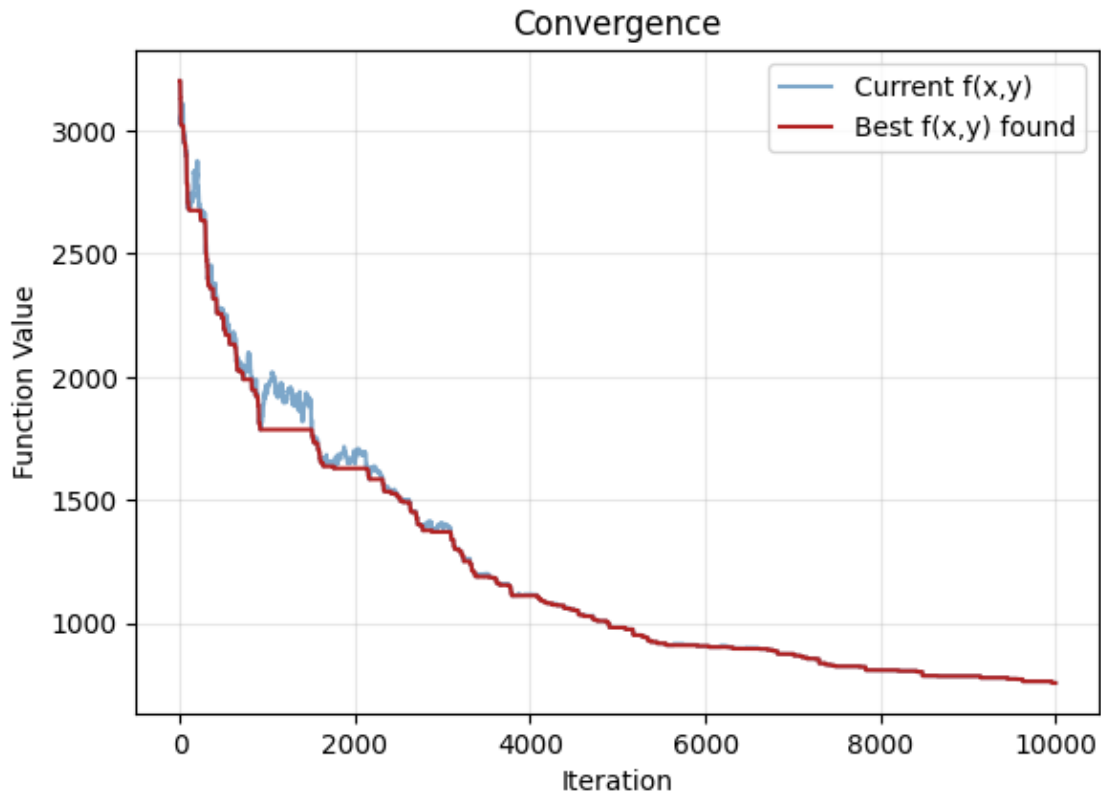
It plateaus almost immediately — $T_k = 20 \times 0.97^k$ is already effectively zero by iteration ~ 500 , regardless of how many more iterations the search is allowed. Extra time alone cannot help once the temperature has already collapsed to zero; the schedule finished cooling long before the budget did. The cooling **rate** itself has to change, not just the iteration count.

```
[120]: random.seed(5972)
s_o = tuple(random.sample(customers, len(customers)))
s_b, F_c, F_b = sa(s_o, f, N, T_o=20, r=0.9995, n=10000, t=1e-9)
z = f(s_b)
print(f"r=0.9995, n={n:<6} -> cost = {z:.2f}    gap = {(z - optimal) / optimal * 100:.1f}%")

fig, ax = plt.subplots(figsize=(6.5, 4.5))
ax.plot(F_c, color="steelblue", linewidth=1.5, alpha=0.7, label="Current f(x,y)")
ax.plot(F_b, color="firebrick", linewidth=1.5, label="Best f(x,y) found")
ax.set_xlabel("Iteration")
ax.set_ylabel("Function Value")
ax.set_title("Convergence")
ax.grid(alpha=0.3)
ax.legend()
```

```
plt.show()
```

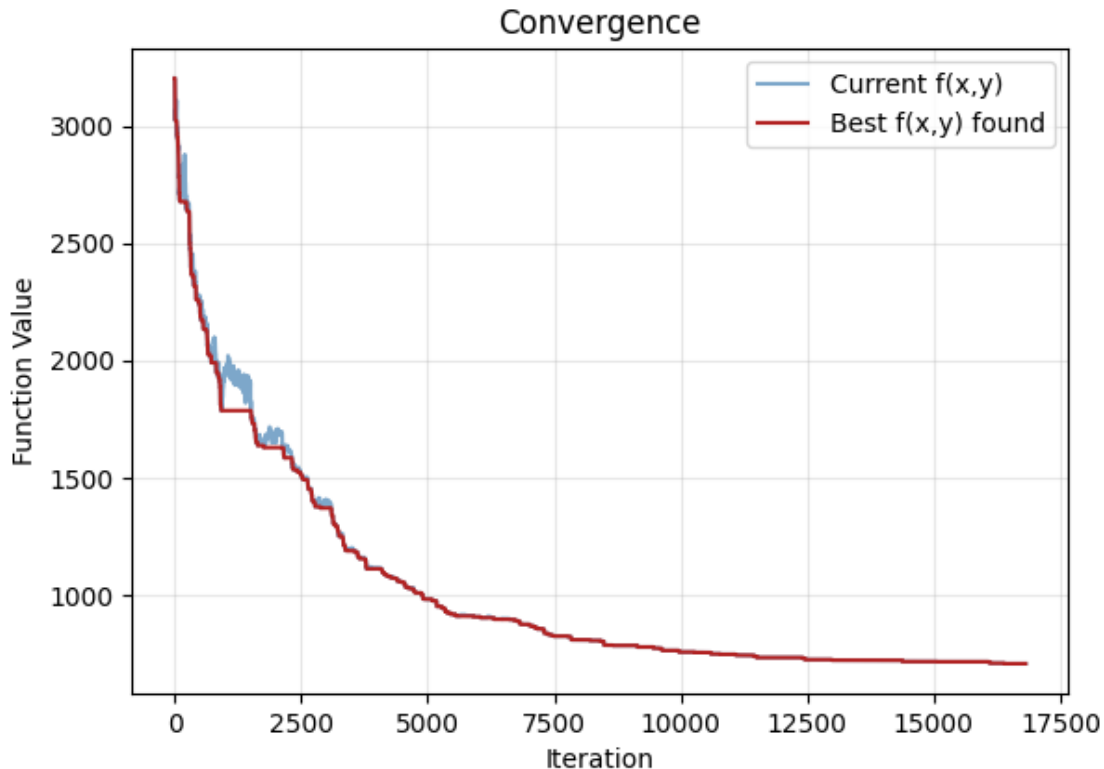
r=0.9995, n=10000 -> cost = 757.65 gap = 20.5%



```
[121]: random.seed(5972)
s_o = tuple(random.sample(customers, len(customers)))
s_b, F_c, F_b = sa(s_o, f, N, T_o=20, r=0.9995, n=20000, t=1e-9)
z = f(s_b)
print(f"r=0.9995, n={n:<6} -> cost = {z:.2f}    gap = {(z - optimal) / optimal * 100:.1f}%")

fig, ax = plt.subplots(figsize=(6.5, 4.5))
ax.plot(F_c, color="steelblue", linewidth=1.5, alpha=0.7, label="Current f(x,y)")
ax.plot(F_b, color="firebrick", linewidth=1.5, label="Best f(x,y) found")
ax.set_xlabel("Iteration")
ax.set_ylabel("Function Value")
ax.set_title("Convergence")
ax.grid(alpha=0.3)
ax.legend()
plt.show()
```

$r=0.9995$, $n=10000$ $\rightarrow$ cost = 707.94 gap = 12.6%



Interpretation. Slowing the cooling rate to $r = 0.9995$ and giving it a budget to match (20,000 iterations, matched to the *slower* schedule, not just “more”) drops the gap from over 100% to well under 20%. Nothing about $r = 0.97$ itself was wrong — it was calibrated correctly, for the budget it was calibrated on. What transferred to the larger instance was the **qualitative** lesson (don’t cool too fast, don’t cool too slow) — Lecture 25’s step-size finding on Ackley made the same kind of point. What did **not** transfer was the literal numeric value of r : a rate tuned to finish cooling inside 1,000 iterations will always finish cooling inside roughly 1,000 iterations, no matter how large the instance or how much extra time you allow, because r controls decay *per iteration*, not decay relative to problem difficulty.

Managerial insight: a calibration performed on a small, cheap-to-experiment-with instance is not a number you can hand off unchanged to a production-scale problem — it is evidence about the *shape* of good behaviour (moderate cooling beats extremes) that still has to be re-scaled to the new problem’s effective budget. Treating a calibrated parameter as a fixed constant, rather than as something coupled to the amount of search effort available, is a common and avoidable mistake.

1.4 In-Class Exercise

1.4.1 Exercise 1 — How Much Rescaling Is Enough?

Sweep $r \in \{0.995, 0.997, 0.999\}$ on `eil101` at a fixed budget of $n = 5000$, and identify which comes closest to the rescaled result above at a fraction of its iteration budget.

```
[122]: for r in [0.995, 0.997, 0.999]:
        random.seed(5972)
        s_o = tuple(random.sample(customers, len(customers)))
        s_b, F_c, F_b = sa(s_o, f, N, T_o=20, r=r, n=5000, t=1e-9)
        z = f(s_b)
        print(f"r={r:<6} n=5000 -> cost = {z:.2f}    gap = {(z - optimal) / optimal_
↪* 100:.1f}%")
```

```
r=0.995  n=5000 -> cost = 1452.46    gap = 130.9%
r=0.997  n=5000 -> cost = 1033.83    gap = 64.4%
r=0.999  n=5000 -> cost = 1224.70    gap = 94.7%
```

Interpretation. All three recover substantially compared to the naive $r = 0.97$ transfer, but not uniformly close to the full-budget result: $r = 0.999$ gets to a 31.2% gap at $n = 5000$ — better than the others, but still well short of the 16.6% gap the same rate reached at $n = 10000$. Half the budget buys real but partial recovery, confirming that the *direction* of the fix (slower cooling) matters even before the *amount* of extra budget is fully in place.

1.5 Take-Away Exercises

1.5.1 Exercise 1 — A Principled Rescaling Rule

Rather than guessing rescaled values by trial and error, propose a rule for how r should change as a function of problem size (hint: consider how many iterations it takes $T_o r^k$ to fall below some small threshold, and how that iteration count might reasonably scale with the number of stops). Test your rule on `eil101`.

1.5.2 Exercise 2 — A Third Instance

Find the coordinates for a third TSPLIB instance of a different size (e.g. `eil176`, roughly halfway between `eil151` and `eil101`), and check whether your Exercise 1 rescaling rule predicts its behaviour reasonably.

1.6 Circling Back

- **Lecture 25 (Simulated Annealing: Algorithm):** the step-size sensitivity found there (Ackley) and the cooling-rate sensitivity found here (TSPLIB) are the same underlying lesson, on two different problems.
- **Lecture 24 (Simulated Annealing: Motivation & Pseudocode):** the hand-traced example's cooling schedule was a toy version of exactly the mechanism calibrated here at real scale.

1.7 Moving Forward

- **Lecture 27 (Genetic Algorithm: Motivation & Pseudocode):** introduces Lecture 23's second paradigm, population search.
-

1.8 Further Reading

- Reinelt, G. (1991). “TSPLIB — A Traveling Salesman Problem Library.” *ORSA Journal on Computing*, 3(4), 376-384.
- Christofides, N. and Eilon, S. (1969). “An Algorithm for the Vehicle-Dispatching Problem.” *Operational Research Quarterly*, 20(3), 309-318 — origin of the `eil` instance family.
- Bertsimas, D. and Tsitsiklis, J. (1993). “Simulated Annealing.” *Statistical Science*, 8(1), 10-15.